

Algoritmos de Caminhos Mínimos em Grafos

Trabalho Prático 3 — Dijkstra, Bellman-Ford e Duan

Isabela M. Andrade Júlia A. da Rocha
Otávio de Oliveira Pedro Brassi Luccas

Departamento de Ciências da Computação
Universidade Federal de Alfenas

Maio de 2026

Problema: dado um grafo ponderado e um vértice de origem, encontrar a menor distância da origem até todos os demais vértices.

Aplicações: GPS, redes de computadores, transporte urbano, jogos digitais, redes logísticas, IA.

Algoritmos implementados:

- **Dijkstra** — abordagem gulosa clássica.
- **Bellman-Ford** — relaxamento global de todas as arestas.
- **Duan** — estratégia híbrida com relaxamentos locais, redução de fronteira e refinamento hierárquico.

Objetivo: comparar empiricamente os três paradigmas em grafos ponderados, conexos e não orientados.

Dijkstra — ideia geral

- Calcula menores caminhos da origem a todos os vértices.
- Dois vetores auxiliares: `dist[]` (distâncias) e `visitado[]` (já finalizados).
- Inicialização: `dist[i] = INT_MAX/2`, `dist[origem] = 0`.
 - A divisão por 2 evita *overflow* nas somas do relaxamento.
- A cada iteração:
 - 1 escolhe vértice não visitado de menor `dist`;
 - 2 marca como visitado;
 - 3 *relaxa* suas arestas: se `dist[u] + peso(u,v) < dist[v]`, atualiza.

Complexidade

$\mathcal{O}(V^2)$ — matriz de adjacência + busca linear (**abordagem gulosa**).

Dijkstra — pseudocódigo

```
Algoritmo Dijkstra(matriz, n, origem)
  para i ← 0 até n-1 faça
    dist[i] ← INFINITO ; visitado[i] ← falso
  fim-para
  dist[origem] ← 0
  para iter ← 0 até n-1 faça
    menor ← INFINITO ; u ← -1
    para i ← 0 até n-1 faça
      se visitado[i] = falso e dist[i] < menor então
        menor ← dist[i] ; u ← i
    fim-se
  fim-para
  se u = -1 então retorna
  visitado[u] ← verdadeiro
  para cada vizinho v de u faça
    se dist[u] + peso(u,v) < dist[v] então
      dist[v] ← dist[u] + peso(u,v)
    fim-se
  fim-para
```

- Resolve SSSP por **relaxamentos sucessivos** de todas as arestas.
- Diferente de Dijkstra: *não* seleciona o menor a cada passo.
- Permite **pesos negativos** (não explorado neste trabalho).
- Inicialização análoga ao Dijkstra (sem vetor visitado).
- **Otimização implementada:** parada antecipada quando nenhuma distância é alterada em uma rodada completa.

Complexidade

$\mathcal{O}(V^3)$ — três laços aninhados (**relaxamento global**).

Bellman-Ford — pseudocódigo

```
Algoritmo Bellman-Ford(matriz, n, origem)
  para i ← 0 até n-1 faça
    dist[i] ← INFINITO
  fim-para
  dist[origem] ← 0
  para k ← 1 até n-1 faça
    mudou ← falso
    para u ← 0 até n-1 faça
      para v ← 0 até n-1 faça
        se matriz[u][v] ≠ 0 então
          se dist[u] + matriz[u][v] < dist[v] então
            dist[v] ← dist[u] + matriz[u][v]
            mudou ← verdadeiro
          fim-se
        fim-se
      fim-para
    fim-para
    se mudou = falso então pare fim-se // parada antecipada
  fim-para
```

Duan — ideia geral

Abordagem moderna que **reduz a dependência da ordenação global** de vértices presente no Dijkstra.

Estratégia: relaxamentos locais, redução de fronteira, seleção de pivôs e refinamento incremental.

Parâmetros fundamentais:

$$k = \lfloor \log^{1/3}(n) \rfloor \quad t = \lfloor \log^{2/3}(n) \rfloor$$

- k — iterações locais de relaxamento (estilo Bellman-Ford);
- t — fator de divisão hierárquica da fronteira.

Estrutura da fronteira: conjuntos S (atual), W (alcançados), W_{prox} (novos), pivots (selecionados).

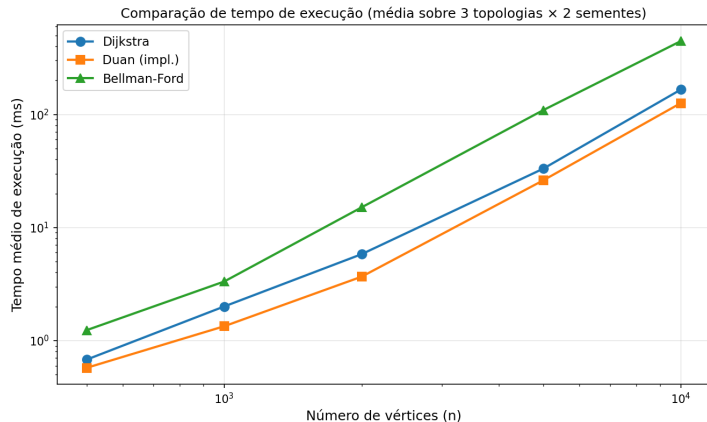
Complexidade

Teórica: $\mathcal{O}(m \log^{2/3} n)$ — **relaxamento local e pivôs**. Na prática, o uso de matriz de

Duan — pseudocódigo simplificado

```
Algoritmo Duan(matriz, n, origem)
    k ←  $\log^{(1/3)} n$  ; t ←  $\log^{(2/3)} n$ 
    para i ← 0 até n-1 faça dist[i] ← INFINITO fim-para
    dist[origem] ← 0
    S ← {origem} ; W ← {origem}
    enquanto S ≠ vazio faça
        // Fase 1 relaxamento local (k iteracoes, estilo Bellman-Ford)
        repete k vezes
            para cada u em W, cada vizinho v de u faça
                se dist[u] + matriz[u][v] < dist[v] então
                    dist[v] ← dist[u] + matriz[u][v]
                    W ← W uniao {v}
            fim-se
        fim-para
        // Fase 2 selecao de pivos
        para cada u em S faça
            se alcance(u) ≥ k ou |S| ≤ 2 então pivots ← pivots uniao {u} fim-se
        fim-para
        // Fase 3 Dijkstra restrito sobre a fronteira (ordem crescente)
```


Resultados — comparação geral



Tempo médio em função de n (escala log-log).

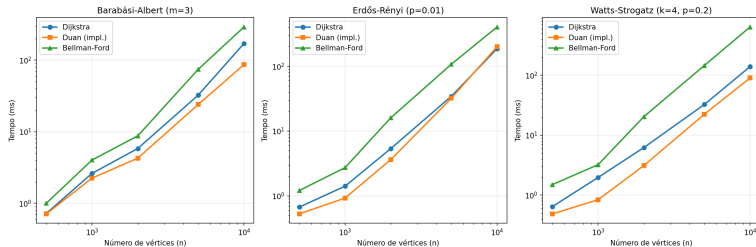
Configuração:

- 30 testes (3 topologias × 5 tamanhos × 2 sementes).
- $n = 500$ a 10 000 vértices.
- Tempo em ms via `clock()`.

Hierarquia clara:

- $BF > Dij > Duan$ em todos os tamanhos.
- Em $n = 10\,000$: Duan é $\sim 24\%$ mais rápido que Dijkstra em média.
- Bellman-Ford é **1 ordem de grandeza** mais lento que os demais.

Resultados — por topologia



Topologias:

- **Barabási** — escala livre (hubs).
- **Erdős** — aleatório uniforme.
- **Watts** — pequeno-mundo.

Diferenças:

- Hierarquia preservada nas três.
- **Barabási**: Duan ganha de até $1,97\times$ sobre Dijkstra.
- **Erdős**: Duan empata ou perde para Dijkstra em $n = 10\,000$.
- **Watts**: BF chega a $4,58\times$ mais lento que Dijkstra.

Razões de crescimento (média das três topologias):

Transição n	Dijkstra	Duan	Bellman-Ford	Esperado (V^2/V^3)
500 $\rightarrow$ 1000	$\times 2,95$	$\times 2,33$	$\times 2,69$	$\times 4 / \times 8$
1000 $\rightarrow$ 2000	$\times 2,90$	$\times 2,74$	$\times 4,53$	$\times 4 / \times 8$
5000 $\rightarrow$ 10000	$\times 5,00$	$\times 4,82$	$\times 4,08$	$\times 4 / \times 8$

- **Dijkstra:** crescimento próximo do quadrático ($\sim \times 4$ ao dobrar n), coerente com $\mathcal{O}(V^2)$.
- **Bellman-Ford:** consistentemente o mais lento; razão BF/Dij cresce de $\sim 1,4\times$ (Barabási, $n = 500$) para $4,58\times$ (Watts, $n = 10\,000$).
- **Duan:** crescimento similar ao Dijkstra; a vantagem vem da **constante menor**, não de um expoente diferente.

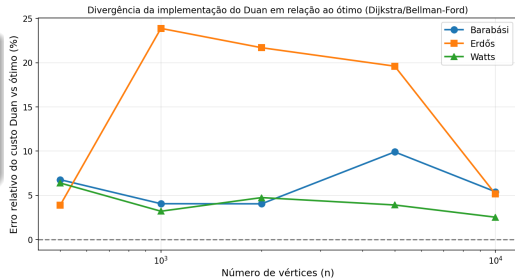
Achado importante

Em **todas as 30 instâncias**, o custo retornado pelo Duan foi *estritamente maior* que o ótimo (Dij = BF), com desvios de +2,52% a +23,88%.

Padrão observado:

- Erro maior em Erdős-Rényi.
- Erro menor em Watts-Strogatz.
- Sugere sensibilidade do FindPivots a topologias com graus heterogêneos.

A implementação atual precisa ser refinada para garantir corretude.



Dijkstra

Gulosa eficiente. Gargalo: busca linear pelo menor vértice ($\mathcal{O}(V^2)$ com matriz). Melhor equilíbrio entre simplicidade e desempenho.

Bellman-Ford

Relaxamentos globais sucessivos. Vantagem teórica (pesos negativos) não explorada. **Consistentemente o mais lento** em todos os testes.

Duan

Remove dependência de ordenação global. Tempos competitivos com Dijkstra — especialmente em Barabási-Albert. Limitações: (i) divergência no custo; (ii) matriz de adjacência mascara ganho assintótico.

- Comparação entre três paradigmas distintos para SSSP em grafos ponderados, conexos e não orientados.
- Resultados experimentais confirmam, em ordem de magnitude, as previsões teóricas:
 - Dijkstra cresce próximo do quadrático;
 - Bellman-Ford é consistentemente o mais lento;
 - Duan apresenta tempos competitivos com Dijkstra, com vantagem em grafos Barabási e Watts de 10 000 vértices.
- Trade-off observado: **simplicidade vs. custo computacional vs. desempenho prático.**
- **Trabalho futuro:** corrigir a finalização do Duan, substituir matriz por lista de adjacência, ampliar topologias e tamanhos.

Obrigado!

Perguntas?

Trabalho Prático 3 — Algoritmos de Caminhos Mínimos em Grafos
UNIFAL-MG — Maio de 2026